DAY 14

NAME: Nandhini V
DATE: 25-08-2025

Task:

- Create a backup commit of the existing book and sales details project.
- Move the queries of the book store from the controller into the Service using Dependency injection
- (optional) Once the architecture is changed to service then implement the repo pattern also

Code:

IBookRepo.cs

```
using Day12Api.Models;
using Day12Api.Services;

namespace Day12Api.Repo
{
   public interface IBookRepo
   {
      void AddSalesInfo(SalesEntry entry);
      CopiesDto GetCopies(string book_name, int year);
      List<SalesByAuthorDto> GetSalesByAuthor(string authname);
      List<SalesDetailDto> GetSalesByAuthorWithJoin(string auth);
      IEnumerable<Book> GetAllBooks();
      List<NewBook> GetAllNewBooks();
      List<BookByAuthorDto> GetBookByAuthor(string name);
      void AddAuthor(Author author);
      void AddNewBook(BookAndAuthorEntry bae);
      void AddBook(Book book);
      void DeleteBook(int id);
      List<Book> GetBooksOrderedByPrice(bool ascending);
      List<Book> GetTop5Books();
      CostDetailsDto GetCostDetails();

   }
}
```

BookRepo.cs

```
using Day12Api.Context;
using Day12Api.Models;
using Day12Api.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace Day12Api.Repo
{
   public class BookRepo : IBookRepo
   {
      private AppDbContext _appDbContext;
      public BookRepo(AppDbContext ctxt)
      {
         _appDbContext = ctxt;
      }

      public void AddSalesInfo(SalesEntry entry)
      {
         var book_to_be_inserted = _appDbContext.NewBooks.Where(b => b.Title ==
entry.Book_name).FirstOrDefault();
         Sales sales = new Sales() { BookId = book_to_be_inserted.NewBookId, No_Of_Copies
= entry.No_Of_Copies, Year = entry.Year };
         _appDbContext.BookSales.Add(sales);
         _appDbContext.SaveChanges();
      }

      public CopiesDto GetCopies(string book_name, int year)
      {
         var copies = _appDbContext.BookSales.Include(s => s.book).ThenInclude(b =>
b.author).Where(bs => bs.book.Title == book_name && bs.Year == year).FirstOrDefault();
         if (copies == null)
            return null;
         return new CopiesDto { BookId = copies.BookId, Author =
copies.book.author.AuthorName, NumberOfCopiesSold = copies.No_Of_Copies, Cost =
copies.book.Price };
      }

      public List<SalesByAuthorDto> GetSalesByAuthor(string authname)
      {
         var sales = _appDbContext.BookSales.Where(a => a.book.author.AuthorName ==
authname).Select(x => new SalesByAuthorDto { BookName = x.book.Title, Year = x.Year,
CopiesSold = x.No_Of_Copies, Cost = x.book.Price, Author = x.book.author.AuthorName
}).ToList();
         if (sales == null)
            return null;
         return sales;
      }
```

```
        public List<SalesDetailDto> GetSalesByAuthorWithJoin(string auth)
        {
            var val_using_join = _appDbContext.BookSales.Join(_appDbContext.NewBooks,
              sales => sales.BookId,
              books => books.NewBookId,
              (sale, book) => new SalesDetailDto { Name = book.Title, Copies =
sale.No_Of_Copies, Year = sale.Year, Author = book.author.AuthorName, BookId =
book.NewBookId }
              ).Where(x => x.Author == auth).ToList();
            if(val_using_join == null)
                return null;
            return val_using_join;
        }

        public IEnumerable<Book> GetAllBooks()
        {
            return _appDbContext.Books.ToList();
        }

        public List<NewBook> GetAllNewBooks()
        {
            return _appDbContext.NewBooks.Include(a => a.author).ToList();
        }

        public List<BookByAuthorDto> GetBookByAuthor(string name)
        {
            var all_books = _appDbContext.NewBooks.Where(a => a.author.AuthorName ==
name).Select(a => new BookByAuthorDto { Author = a.author.AuthorName, BookName =
a.Title, Price = a.Price, NoOfSold = a.No_Of_Sold, Sales = (a.Price * a.No_Of_Sold) }).ToList();
            if(all_books == null)
                return null;
            return all_books;
        }

        public void AddAuthor(Author author)
        {
            _appDbContext.Authors.Add(author);
            _appDbContext.SaveChanges();
        }

        public void AddNewBook(BookAndAuthorEntry bae)
        {
            var author = _appDbContext.Authors.Where(a => a.AuthorName ==
bae.AuthorName).FirstOrDefault();
            NewBook newBook = new NewBook() { Title = bae.BookTitle, Price = bae.Price,
AuthorId = author.AuthorId };
            _appDbContext.NewBooks.Add(newBook);
            _appDbContext.SaveChanges();
        }
```

```
    public void AddBook(Book book)
    {
        _appDbContext.Books.Add(book);
        _appDbContext.SaveChanges();
    }

    public void DeleteBook(int id)
    {
        var book_to_delete = _appDbContext.Books.FirstOrDefault(b => b.Id == id);
        _appDbContext.Books.Remove(book_to_delete);
        _appDbContext.SaveChanges();
    }

    public List<Book> GetBooksOrderedByPrice(bool ascending)
    {
        return ascending
            ? _appDbContext.Books.OrderBy(b => b.Price).ToList()
            : _appDbContext.Books.OrderByDescending(b => b.Price).ToList();
    }

    public List<Book> GetTop5Books()
    {
        return _appDbContext.Books
            .OrderByDescending(b => b.Price)
            .Take(5)
            .ToList();
    }

    public CostDetailsDto GetCostDetails()
    {
        var totalPrice = _appDbContext.Books.Sum(b => b.Price);
        var avgPrice = _appDbContext.Books.Average(b => b.Price);
        return new CostDetailsDto { TotalPrice = totalPrice, AveragePrice = avgPrice };
    }
}
public class CopiesDto
{
    public int BookId { get; set; }
    public string Author { get; set; }
    public int NumberOfCopiesSold { get; set; }
    public decimal Cost { get; set; }
}

public class SalesByAuthorDto
{
    public string BookName { get; set; }
    public int Year { get; set; }
    public int CopiesSold { get; set; }
    public decimal Cost { get; set; }
    public string Author { get; set; }
}
```

```
    public class SalesDetailDto
    {
        public string Author { get; set; }
        public string Name { get; set; }
        public int Copies { get; set; }
        public int Year { get; set; }
        public int BookId { get; set; }
    }
    public class BookByAuthorDto
    {
        public string Author { get; set; }
        public string BookName { get; set; }
        public decimal Price { get; set; }
        public int NoOfSold { get; set; }
        public decimal Sales { get; set; }
    }
    public class CostDetailsDto
    {
        public decimal TotalPrice { get; set; }
        public double AveragePrice { get; set; }
    }
}
```

IBookService.cs

```
using Day12Api.Models;
using Day12Api.Repo;
using Microsoft.AspNetCore.Mvc;

namespace Day12Api.Services
{
    public interface IBookService
    {
        void AddSalesInfo(SalesEntry entry);
        CopiesDto GetCopies(string book_name, int year);
        List<SalesByAuthorDto> GetSalesByAuthor(string authname);
        List<SalesDetailDto> GetSalesByAuthorWithJoin(string auth);
        IEnumerable<Book> GetAllBooks();
        List<NewBook> GetAllNewBooks();
        List<BookByAuthorDto> GetBookByAuthor(string name);
        void AddAuthor(Author author);
        void AddNewBook(BookAndAuthorEntry bae);
        void AddBook(Book book);
        void DeleteBook(int id);
        List<Book> GetBooksOrderedByPrice(bool ascending);
        List<Book> GetTop5Books();
        CostDetailsDto GetCostDetails();
    }
}
```

BookStore.cs

```
using Day12Api.Context;
using Day12Api.Models;
using Day12Api.Repo;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace Day12Api.Services
{
   public class BookStore : IBookService
   {
      private AppDbContext _appDbContext;
      private IBookRepo bookRepo;
      public BookStore(AppDbContext ctxt, IBookRepo _bookRepo)
      {
         _appDbContext = ctxt;
         bookRepo = new BookRepo(_appDbContext);
      }

      public void AddSalesInfo(SalesEntry entry)
      {
         bookRepo.AddSalesInfo(entry);
      }

      public CopiesDto GetCopies(string book_name, int year)
      {
         return bookRepo.GetCopies(book_name, year);
      }

      public List<SalesByAuthorDto> GetSalesByAuthor(string authname)
      {
         return bookRepo.GetSalesByAuthor(authname);
      }

      public List<SalesDetailDto> GetSalesByAuthorWithJoin(string auth)
      {
         return bookRepo.GetSalesByAuthorWithJoin(auth);
      }

      public IEnumerable<Book> GetAllBooks()
      {
         return bookRepo.GetAllBooks();
      }

      public List<NewBook> GetAllNewBooks()
      {
         return bookRepo.GetAllNewBooks();
      }
```

```
        public List<BookByAuthorDto> GetBookByAuthor(string name)
        {
          return bookRepo.GetBookByAuthor(name);
        }

        public void AddAuthor(Author author)
        {
           bookRepo.AddAuthor(author);
        }

        public void AddNewBook(BookAndAuthorEntry bae)
        {
           bookRepo.AddNewBook(bae);
        }

        public void AddBook(Book book)
        {
           bookRepo.AddBook(book);
        }

        public void DeleteBook(int id)
        {
           bookRepo.DeleteBook(id);
        }

        public List<Book> GetBooksOrderedByPrice(bool ascending)
        {
           return bookRepo.GetBooksOrderedByPrice(ascending);
        }

        public List<Book> GetTop5Books()
        {
          return bookRepo.GetTop5Books();
        }

        public CostDetailsDto GetCostDetails()
        {
           return bookRepo.GetCostDetails();
        }
    }

}
```

BookController.cs

```
using Day12Api.Context;
using Day12Api.Migrations;
using Day12Api.Models;
```

```
using Day12Api.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace Day12Api.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class BookController : Controller
    {
        IBookService bookInstance;
        public BookController(IBookService _bookInstance)
        {
            bookInstance = _bookInstance;
        }

        [HttpPost("InsertSales")]
        public IActionResult AddSalesInfo(SalesEntry entry)
        {
            bookInstance.AddSalesInfo(entry);
            return Ok("Sales data inserted");
        }

        [HttpGet("GetCopies")]
        public IActionResult GetCopies(string book_name, int year)
        {
            var copies = bookInstance.GetCopies(book_name, year);
            if (copies == null)
                return NotFound("No sales data found for this book and year");
            return Ok(copies);
        }

        [HttpPost("GetSalesByAuthor")]
        public IActionResult GetSalesByAuthor(string authname)
        {
            var result = bookInstance.GetSalesByAuthor(authname);
            if (result == null)
                return NotFound("No sales data found for this author");
            return Ok(result);
        }

        [HttpGet("GetSalesByAuthorWithJoin")]
        public IActionResult GetSalesByAuthorWithJoin(string auth)
        {
            var final_res = bookInstance.GetSalesByAuthorWithJoin(auth);
            if (final_res == null)
                return NotFound("Author not found");
            return Ok(final_res);
        }
```

```
[HttpGet("GetAllBooks")]
public IActionResult GetAllBooks()
{
    var books = bookInstance.GetAllBooks();
    if (books == null)
        return NotFound("No books found");
    return Ok(books);
}

[HttpGet("FetchAllNewBooks")]
public IActionResult GetAllNewBooks()
{
    var allNewBooks = bookInstance.GetAllNewBooks();
    if (allNewBooks == null)
        return NotFound("No new books found");
    return Ok(allNewBooks);
}

[HttpGet("FetchByAuthor")]
public IActionResult GetBookByAuthor(string name)
{
    var result = bookInstance.GetBookByAuthor(name);
    if (result == null)
        return NotFound("No books found for this author");
    return Ok(result);
}

[HttpPost("AddAuthor")]
public IActionResult AddAuthor(Author author)
{
    bookInstance.AddAuthor(author);
    return Ok("Author added");
}

[HttpPost("AddNewBook")]
public IActionResult AddNewBook(BookAndAuthorEntry bae)
{
    bookInstance.AddNewBook(bae);
    return Ok("Book added");
}

[Authorize]
[HttpPost("AddBook")]
public IActionResult AddBook(Book book)
{
    bookInstance.AddBook(book);
    return Ok("Added book to the store");
}

[Authorize(Roles = "Admin")]
[HttpDelete("RemoveBook")]
```

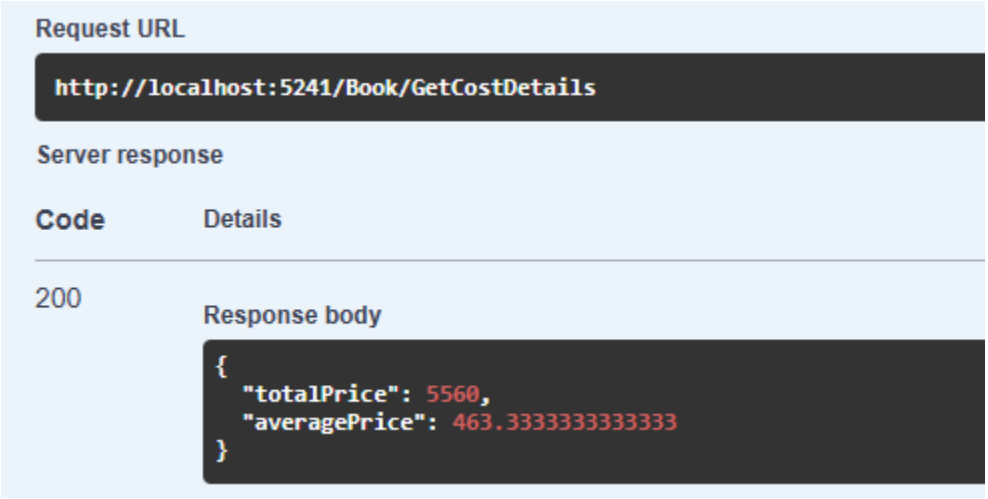
```
        public IActionResult DeleteBook(int id)
        {
            bookInstance.DeleteBook(id);
            return Ok("Deleted");
        }

        [HttpPost("GetBooksOrderedByPrice")]
        public IActionResult GetBooksOrderedByPrice(bool ascending)
        {
            var books = bookInstance.GetBooksOrderedByPrice(ascending);
            return Ok(books);
        }

        [HttpGet("GetTop5Books")]
        public IActionResult GetTop5Books()
        {
            var books = bookInstance.GetTop5Books();
            return Ok(books);
        }

        [HttpGet("GetCostDetails")]
        public IActionResult GetCostDetails()
        {
            var costDetails = bookInstance.GetCostDetails();
            return Ok(costDetails);
        }
    }
}
```

Output:



Request URL

```
http://localhost:5241/Book/GetSalesByAuthor?authname=ruskin%20bond
```

Server response

Code	Details
200	Response body


```
[
  {
    "bookName": "The Room on The Roof",
    "year": 2025,
    "copiesSold": 30,
    "cost": 250,
    "author": "Ruskin Bond"
  },
  {
    "bookName": "The Room on The Roof",
    "year": 2024,
    "copiesSold": 20,
    "cost": 250,
    "author": "Ruskin Bond"
  },
  {
    "bookName": "Cherry Tree",
    "year": 2024,
    "copiesSold": 12,
    "cost": 200,
    "author": "Ruskin Bond"
  }
]
```


Request URL

```
http://localhost:5241/Book/GetCopies?book_name=harry%20potter%201&year=2023
```

Server response

Code	Details
200	Response body


```
{
  "bookId": 1,
  "author": "JK Rowling",
  "numberOfCopiesSold": 10,
  "cost": 300
}
```